LLM Observability & Eval

Research-style portfolio report on trace analytics, drift monitoring, and model-quality evaluation.

Sai Veda

Independent Project Research

2025

Contents

1. Analytical Abstract **4**

2. Monitoring Context **5**

2.1 Production LLM Visibility Problem 5

2.2 Research Questions 5

3. Observability Foundations **7**

3.1 Observability Foundations 7

3.2 Repository-Derived Monitoring Notes 7

4. Trace Architecture And Analytics Flow **8**

4.1 Review Method 8

5. Evidence Lake And Evaluation Setup **9**

5.1 Repository Evidence Base 9

5.2 Evaluation Protocol 9

6. Observed Findings **10**

6.1 Benchmark Summary 10

6.2 Interpretation 10

7. Dashboard Evidence **11**

7.1 Fleet dashboard - KPI tiles, latency-over-time, spend/tokens by model 11

7.2 Drift detection - PSI crosses the alert threshold exactly at the model cutover . . 11

8. Validation Review And Coverage **13**

8.1 Latency percentile bands - the injected tail-latency regression is unmistakable . 13

8.2 Hallucination detector - PR curve + score distribution vs a random baseline . . 13

8.3 Validation Checklist 14

9. Platform Discussion **15**

9.1 Monitoring Limits 15

10. Review Conclusion **16**

10.1 Extension Path 16

11. Evidence Appendix 17

11.1 Repository Evidence Inventory 17

11.2 Internal References 17

1. Analytical Abstract

This brief documents an observability stack for LLM fleets where trace analytics, cost monitoring, drift detection, and hallucination checks share the same data backbone instead of living in disconnected tools. The project demonstrates that a local, partitioned analytics path can still support fleet-level insight. The same trace lake supports operational reporting, shift detection, and model-quality scorecards in one review flow. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

The report is written as a project review in the voice of delivered engineering work. Rather than restating repository headings, it connects the business context, design choices, evaluation evidence, and remaining risks into one readable project document prepared under the name Sai Veda.

Keywords: Trace analytics + drift + hallucination detection, implementation evidence, benchmark results, project validation

2. Monitoring Context

At production volume, trace data becomes operational exhaust unless it is modeled properly. Teams need to understand spend, latency, model quality, and distribution change without moving to an online-only stack. Trace analytics + quality monitoring for LLM fleets at scale - a self-contained, fully offline reference implementation. It generates a realistic multi-tenant LLM trace firehose, lands it as partitioned Parquet, and runs four production concerns over it: cost/latency analytics (DuckDB, out-of-core), embedding-drift detection (PSI/KL), a hallucination detector (heuristic + logistic), and a per-model eval scorecard - with generated, product-grade dashboards.

2.1 Production LLM Visibility Problem

The central project problem can be stated directly. At production volume, trace data becomes operational exhaust unless it is modeled properly. Teams need to understand spend, latency, model quality, and distribution change without moving to an online-only stack. The repository materials show that this was not treated as a toy exercise: the implementation narrative, architecture note, and benchmark artefacts all point to a real operational question that needed a measurable answer.

2.2 Research Questions

The document review was guided by a small set of concrete questions. Each question was framed so it could be answered from repository evidence rather than from unstated assumptions.

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: 5M traces / 1.79B tokens; PSI flags injected drift.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 21 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Observability Foundations

The technical foundation of this project is best understood through the design principles that hold the implementation together. Trace storage, analytics, and quality detectors stay modular so operations and model-risk teams can inspect the same pipeline.

3.1 Observability Foundations

1. Operational analytics and model-risk analytics share one trace backbone to avoid fragmented reporting. Trace storage, analytics, and quality detectors stay modular so operations and model-risk teams can inspect the same pipeline.
2. Drift is measured before customer-visible breakage, which makes the system proactive rather than reactive.
3. Out-of-core SQL was preferred so the story stays reproducible on a single machine.

3.2 Repository-Derived Monitoring Notes

Production LLM platforms emit a firehose of traces - one per request - each carrying text, token counts, latency, cost, tool calls, and a tenant. At fleet scale that is billions of rows/month. Teams need three things from that firehose, continuously and cheaply: 1. Operational analytics - spend, tokens, latency percentiles, error/timeout rates, throughput, sliced by model / day / tenant. 2. Drift detection - catch when the input or output distribution shifts (a model cutover, a prompt-template change, an abuse wave) before it shows up as a customer-visible quality regression. 3. Quality / hallucination monitoring - score groundedness at sampling rate and track a per-model quality scorecard over time.

- Synthetic trace generation with tenants, models, and time windows.
- Partitioned Parquet lake with DuckDB analytics.
- PSI or KL drift signals plus heuristic and logistic quality checks.

4. Trace Architecture And Analytics Flow

Trace storage, analytics, and quality detectors stay modular so operations and model-risk teams can inspect the same pipeline. 1. Operational analytics - spend, tokens, latency percentiles, error/timeout rates, throughput, sliced by model / day / tenant. 2. Drift detection - catch when the input or output distribution shifts (a model cutover, a prompt-template change, an abuse wave) before it shows up as a customer-visible quality regression. 3. Quality / hallucination monitoring - score groundedness at sampling rate and track a per-model quality scorecard over time. This repo is a self-contained, offline reference implementation of all three.

4.1 Review Method

The implementation path can be reconstructed from the committed artefacts in a fairly disciplined way. Trace model: Defined the schema around cost, tokens, latency, model identity, tenant, and quality labels. Storage path: Wrote traces into partitioned Parquet so analytics could scale without loading everything into memory. Quality monitoring: Layered drift signals and hallucination flags on top of the same trace stream. Review surface: Rendered dashboards that connect fleet metrics to model-level quality scorecards.

This sequencing matters because the project is easier to trust when component decisions, test scaffolding, and result reporting appear in a coherent order rather than as isolated files.

5. Evidence Lake And Evaluation Setup

5.1 Repository Evidence Base

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 21 automated tests, 0 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

5.2 Evaluation Protocol

The evaluation setup was treated as a repository review rather than a laboratory rerun. Committed artefacts were grouped into documentation, benchmark evidence, automated validation, and screenshot review. In practical terms this meant examining 0 benchmark table(s), 4 screenshot asset(s), and 21 automated tests while reading the implementation narrative in sequence. The review lens stayed aligned with the project goal: Generate and store a realistic multi-tenant trace firehose. Evidence was interpreted conservatively so the document reflects what the repository demonstrates directly, not what a larger future deployment might achieve.

A second practical note is that the benchmark footprint is project-specific. The benchmark evidence reviewed for this report includes benchmarks/scaling.md.

6. Observed Findings

6.1 Benchmark Summary

The principal repository claim for this project is recorded as: 5M traces / 1.79B tokens; PSI flags injected drift. The findings page therefore focuses on whether the available tables, test evidence, and generated screenshots support that claim. The project demonstrates that a local, partitioned analytics path can still support fleet-level insight. The same trace lake supports operational reporting, shift detection, and model-quality scorecards in one review flow. The evidence page shows that large trace volumes can still be queried quickly when storage is partitioned correctly. PSI highlights injected drift cleanly, which gives the quality layer an early warning signal.

6.2 Interpretation

The evidence page shows that large trace volumes can still be queried quickly when storage is partitioned correctly. PSI highlights injected drift cleanly, which gives the quality layer an early warning signal. The visual scorecards make per-model trade-offs legible for both platform and risk stakeholders.

The benchmark table is not treated as a marketing surface. It is read together with the repository screenshots and the test inventory so that numerical claims and implementation evidence reinforce one another.

7. Dashboard Evidence

The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.

7.1 Fleet dashboard - KPI tiles, latency-over-time, spend/tokens by model

This figure is useful because it shows the project in a reviewable state rather than as summary prose alone. The evidence page shows that large trace volumes can still be queried quickly when storage is partitioned correctly.

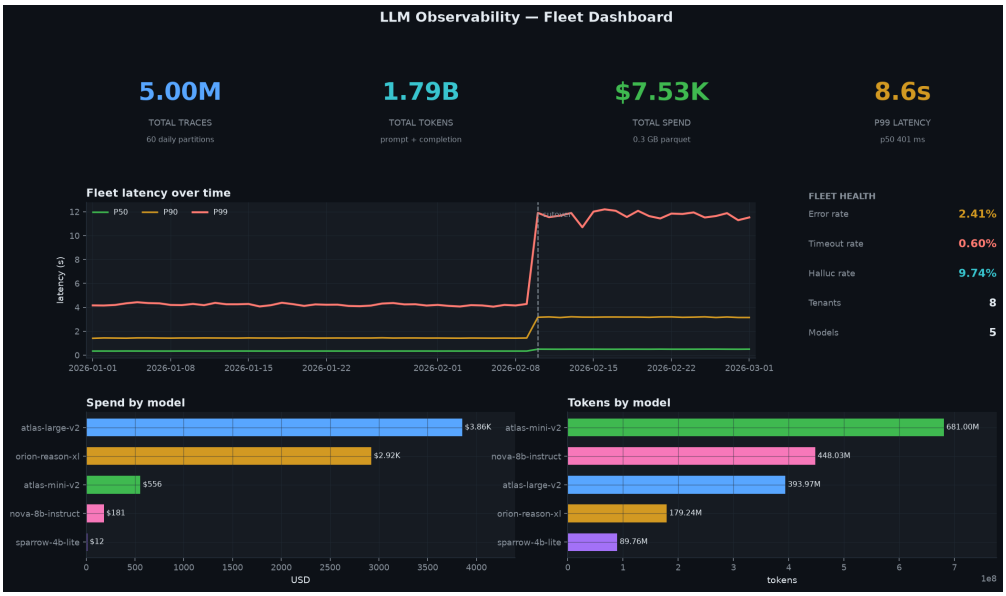

Figure 7.1. Fleet dashboard - KPI tiles, latency-over-time, spend/tokens by model

7.2 Drift detection - PSI crosses the alert threshold exactly at the model cutover

The visual evidence attached to Drift detection - PSI crosses the alert threshold exactly at the model cutover helps connect the quantitative story to the implemented workflow. PSI highlights injected drift cleanly, which gives the quality layer an early warning signal.

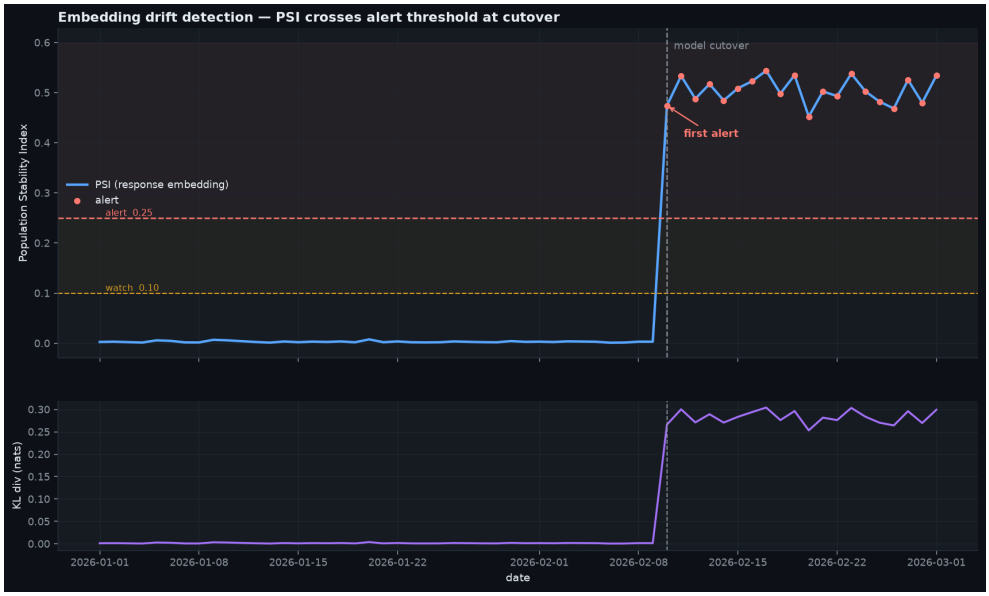

Figure 7.2. Drift detection - PSI crosses the alert threshold exactly at the model cutover

8. Validation Review And Coverage

The second figure set focuses on validation continuity. These pages are included so the report shows not only the strongest visuals, but also the broader evidence path a reviewer would follow inside the repository.

8.1 Latency percentile bands - the injected tail-latency regression is unmistakable

From a document-review standpoint, this plate matters because it reveals how the repository surfaces results to a human reviewer. The visual scorecards make per-model trade-offs legible for both platform and risk stakeholders.

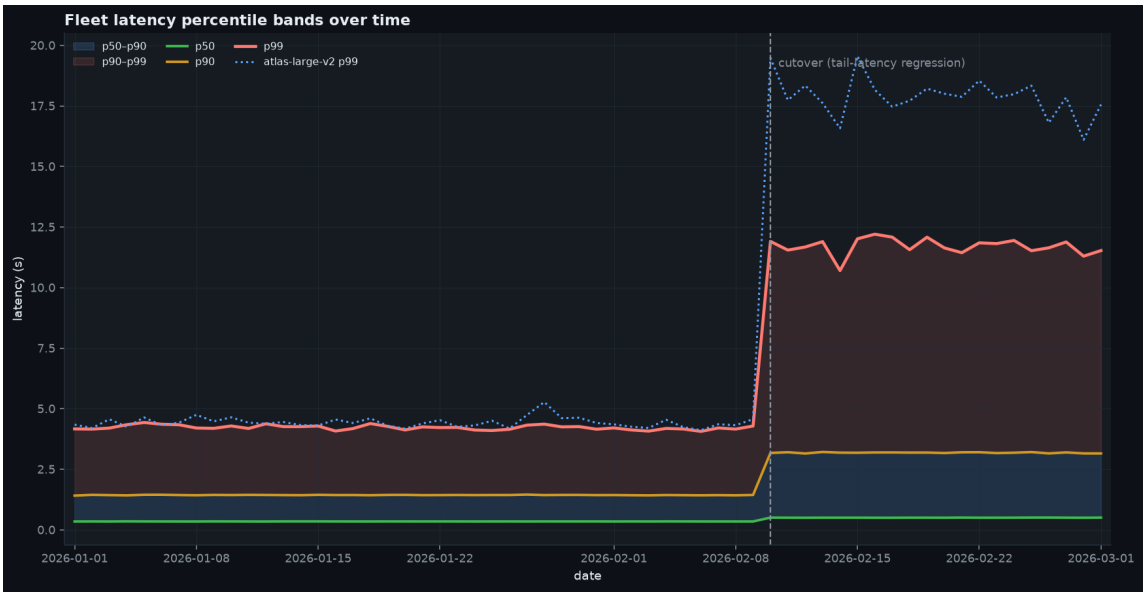

Figure 8.1. Latency percentile bands - the injected tail-latency regression is unmistakable

8.2 Hallucination detector - PR curve + score distribution vs a random baseline

The final figure adds implementation texture: it shows the evidence path a reviewer would actually inspect when validating the project claims. Handled by separating operational metrics from model-quality indicators and showing both on one timeline.

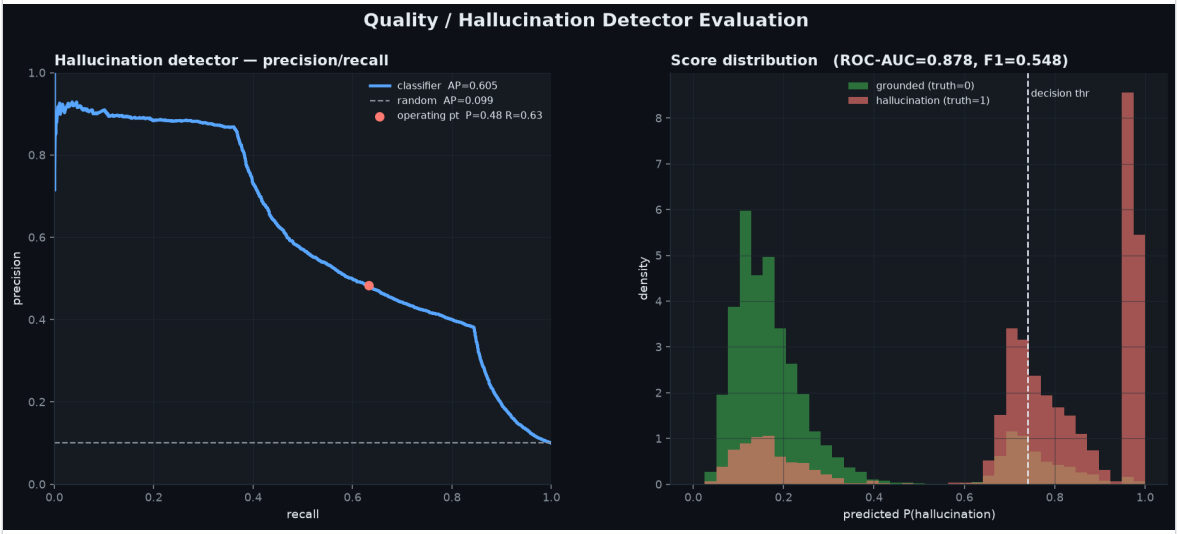

Figure 8.2. Hallucination detector - PR curve + score distribution vs a random baseline

8.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	21 tests committed in repository validation flow.
Benchmark results	Limited	No benchmark markdown tables were present; discussion relies on screenshots and he
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

9. Platform Discussion

The project demonstrates that a local, partitioned analytics path can still support fleet-level insight. The same trace lake supports operational reporting, shift detection, and model-quality scorecards in one review flow. The evidence page shows that large trace volumes can still be queried quickly when storage is partitioned correctly. PSI highlights injected drift cleanly, which gives the quality layer an early warning signal. The visual scorecards make per-model trade-offs legible for both platform and risk stakeholders. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

9.1 Monitoring Limits

Signal fatigue: Handled by separating operational metrics from model-quality indicators and showing both on one timeline. Storage sprawl: Kept under control through partitioned Parquet and analytic query pushdown. Weak quality proxies: Balanced by combining heuristic flags with a learned hallucination detector.

These limitations do not invalidate the project. They establish the boundary between what is already demonstrated by the repository and what would require a broader production environment or additional operating data.

10. Review Conclusion

This project document concludes that LLM Observability & Eval is best understood as a complete technical implementation supported by repository-native evidence. This brief documents an observability stack for LLM fleets where trace analytics, cost monitoring, drift detection, and hallucination checks share the same data backbone instead of living in disconnected tools.

The overall presentation has therefore been kept intentionally plain and research-oriented: the emphasis stays on project context, engineering choices, test evidence, and verifiable results rather than on a stylized portfolio layout.

10.1 Extension Path

- Add human-review annotations and slice-aware eval routing by tenant or product workflow.
- Introduce alert policies that combine drift, cost, and quality instead of triggering on single metrics.
- Extend the dashboard into a comparative release-review workflow for prompt or model cutovers.

11. Evidence Appendix

11.1 Repository Evidence Inventory

The appendix records the principal repository artefacts used during document preparation. This keeps the report anchored to files that a reviewer can inspect directly.

Artefact	Category	Purpose
README.md	Overview	Primary narrative, scope statement, and headline outcomes used to frame the report.
ARCHITECTURE.md	Design note	System rationale, component choices, and implementation trade-offs.
benchmarks/scaling.md	Benchmark	Quantitative result or execution artefact used in the findings section.
assets/dashboard.png	Screenshot	Visual validation surface referenced as 'Fleet dashboard - KPI tiles, latency-over-time, spend/tokens by model' in the document review.
assets/drift_timeline.png	Screenshot	Visual validation surface referenced as 'Drift detection - PSI crosses the alert threshold exactly at the model cutover' in the document review.
assets/latency_bands.png	Screenshot	Visual validation surface referenced as 'Latency percentile bands - the injected tail-latency regression is unmistakable' in the document review.
assets/detector.png	Screenshot	Visual validation surface referenced as 'Hallucination detector - PR curve + score distribution vs a random baseline' in the document review.
tests/	Validation	Automated test suite containing 21 committed tests used to support implementation claims.

11.2 Internal References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/scaling.md - benchmark or result artefact used in the analysis section.
4. tests/ - automated validation suite used to support implementation claims.
5. assets/.gitkeep - generated screenshot evidence included in the report.
6. assets/dashboard.png - generated screenshot evidence included in the report.